

Real-world (config + logging together) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: its own folder, `practice/config_logging_wiring/`, because it needs more than one file:

```
practice/config_logging_wiring/
├─ exceptions.py      MissingConfigError
├─ config.py          load_config() -> Config
├─ logging_setup.py   get_logger(name)
├─ main.py            wires the two together
└─ .env               you create this; never committed
```

Run it: `cd practice/config_logging_wiring && python main.py` — from inside the folder, so `from config import load_config` finds the file sitting next to it.

Builds on: `practice/logging_practice.py` ([Intermediate part 2](#)) — its `get_logger()` is **copied** into `logging_setup.py` below, unchanged. And `practice/custom_errors_practice.py` ([Basic](#)) — the same custom-error pattern, re-written here as `MissingConfigError`. Copies, not imports: this folder runs on its own.

How the approaches below are organised: all four files are shown complete in **Basic Approach 1**, in the order you create them, deliberately without type hints yet — matching the Basic level of every exercise before this one. **Intermediate** adds type hints to `exceptions.py` and `config.py` (shown again in full there) and changes `main.py`; `logging_setup.py` never changes, since it's a verbatim copy of the already-typed `get_logger()` from the logger exercise.

Basic Version

Approach 1 — the direct way

The file you create first, because nothing runs without it — your own `.env`, holding a fake key for now:

```
# practice/config_logging_wiring/.env
OPENAI_API_KEY=sk-test-123
LOG_LEVEL=DEBUG
```

Then the error class — one line of real content, the same pattern as the Basic exercise:

```
# practice/config_logging_wiring/exceptions.py
class MissingConfigError(Exception):
    pass
```

Then the config loader:

```
# practice/config_logging_wiring/config.py
import os

from dotenv import load_dotenv

from exceptions import MissingConfigError

class Config:
    def __init__(self, openai_api_key, log_level):
        self.openai_api_key = openai_api_key
        self.log_level = log_level

    def require_env(key):
        value = os.getenv(key)
        if value is None or value == "":
            raise MissingConfigError("Required environment variable is missing: " + k
        return value

    def load_config():
        load_dotenv()
        api_key = require_env("OPENAI_API_KEY")
        log_level = os.getenv("LOG_LEVEL", "INFO")
        return Config(openai_api_key=api_key, log_level=log_level)
```

A plain class with `__init__`, no type hints yet — same level as the Basic exercises before this one. Type hints get added in Intermediate below, and `@dataclass` (which auto-generates `__init__` for you) is introduced later still, in the Build Task, once you've written this constructor by hand at least once.

Then the logger — this is `get_logger()` copied straight out of `practice/logging_practice.py`, name and signature untouched:

```
# practice/config_logging_wiring/logging_setup.py
# Copied from practice/logging_practice.py (Intermediate part 2), unchanged.
```

```
import logging

def get_logger(name: str) -> logging.Logger:
    """Create (or reuse) a logger with a screen handler and a file handler.

    Safe to call more than once — if this logger already has handlers,
    they are not added again.
    """
    logger = logging.getLogger(name)
    if logger.handlers:
        return logger

    logger.setLevel(logging.DEBUG)

    screen_handler = logging.StreamHandler()
    screen_handler.setLevel(logging.INFO)
    logger.addHandler(screen_handler)

    file_handler = logging.FileHandler("app.log")
    file_handler.setLevel(logging.DEBUG)
    logger.addHandler(file_handler)

    return logger
```

And last, the file this exercise is really about — the wiring:

```
# practice/config_logging_wiring/main.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger("main")

logger.info("Loaded config with log level: " + config.log_level)
```

Expected output (terminal), with the `.env` above:

```
Loaded config with log level: DEBUG
```

The exact text depends on what's actually in your `.env` — this shows the shape of the output, not a fixed value. The copied `get_logger()` also attaches a file handler, so the same line is appended to `practice/config_logging_wiring/app.log`; that file appearing after your first run is expected, not a mistake. If `load_config()` fails (a required setting is missing), this version crashes with a raw Python traceback — fine for a first working version, but not what you'd want a user to see.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`practice/config_logging_wiring/logging_setup.py` stays exactly as shown in Basic Approach 1 above — it's already a verbatim copy of the typed `get_logger()` from the Intermediate logger exercise, so there's nothing to add. `exceptions.py` and `config.py` now get type hints:

```
# practice/config_logging_wiring/exceptions.py
class MissingConfigError(Exception):
    pass
```

```
# practice/config_logging_wiring/config.py
import os

from dotenv import load_dotenv

from exceptions import MissingConfigError

class Config:
    def __init__(self, openai_api_key: str, log_level: str) -> None:
        self.openai_api_key = openai_api_key
        self.log_level = log_level

    def require_env(key: str) -> str:
        value = os.getenv(key)
        if value is None or value == "":
            raise MissingConfigError(f"Required environment variable is missing: {key}")
        return value

    def load_config() -> Config:
        load_dotenv()
        api_key = require_env("OPENAI_API_KEY")
        log_level = os.getenv("LOG_LEVEL", "INFO")
        return Config(openai_api_key=api_key, log_level=log_level)
```

Both `main.py` approaches below keep the same typed `exceptions.py` and `config.py` above.

Approach 1 — f-string, no failure handling yet

```
# practice/config_logging_wiring/main.py
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output (terminal), with the `.env` from Basic Approach 1:

```
Loaded config with log level: DEBUG
```

Approach 2 — with the startup failure handled

If config loading can fail, catch that and stop the program with a clear message instead of a scary error dump.

```
# practice/config_logging_wiring/main.py
import sys

from config import load_config
from exceptions import MissingConfigError
from logging_setup import get_logger

try:
    config = load_config()
except MissingConfigError as e:
    print(f"Startup failed: {e}")
    sys.exit(1)

logger = get_logger(__name__)

logger.info(f"Loaded config with log level: {config.log_level}")
```

Expected output when config is valid: identical to Approach 1.

Expected output when `OPENAI_API_KEY` is missing from `.env` (or set to an empty string):

```
Startup failed: Required environment variable is missing: OPENAI_API_KEY
```

and the process exits with status code `1` — no traceback, no logger involved (there isn't one yet at this point).

Difference from Basic: `exceptions.py` stays a bare `pass`, but `config.py` gets full type hints (`key: str`, `-> Config`, `-> None`), turning every signature into documentation. Both `main.py` approaches also switch to an f-string and `get_logger(__name__)` (so the logger is named after whichever module actually created it, not a hardcoded `"main"`). Approach 2 additionally wraps the one call that can fail — `load_config()` — in a `try / except`, so a missing setting produces one clear line and a clean exit instead of a multi-line traceback a non-technical user would have to decode.

Which one should you actually write? Intermediate Approach 2 (try/except around `load_config()`, clean exit) is completely correct and what most people should default to — it satisfies "handle the startup failure" without extra machinery.

What carries into the Build Task: these same four files, one level more serious. Copy `practice/config_logging_wiring/` to `practice/build_task/` and upgrade from there — the [Build Task solution](#) picks up exactly where this leaves off.